

WDR Bench User Manual

Everything needed to set up, wire, run, test and reflash the ESP32 flight-data replay bench. The command language is summarised here. The **WDR Manual and Protocol** PDF has the full rules.

1 What the bench does

The bench stores a **profile**: a recorded flight turned into rows of servo pulse widths. It plays the profile into up to 4 servos at a fixed frame rate and loops it for an endurance test. It counts every **cycle** and every running hour, and keeps those counts, and the profile itself, through a power cut. A controller on the same Wi-Fi network drives it with short text commands over TCP port 3333.

SPEC	VALUE
Board	ESP32 DevKit (ESP32-D0WD, 4 MB flash, CP210x USB) or the 7semi ESP32-S3 DevKit
Outputs	4 servo PWM channels, 50 Hz, 500–2500 µs, measured accurate to ±2 µs
Profile	Up to 8000 frames at 10–100 frames/s (160 s at 50/s, 13 min at 10/s), saved in flash
Control link	Wi-Fi station, TCP server on port 3333 , one controller at a time
Debug link	USB serial at 115200 baud: the same commands, plus boot messages
Counters	Lifetime cycles, run seconds and active seconds per channel, saved at least once a minute

2 Quick start

- 1 Network.** Turn on a 2.4 GHz hotspot or router named `WELKINRIM-WDR` with password `welkinrim2026`. (To use another network, see section 11.)
- 2 Power.** Plug the board's USB into a laptop or USB charger. It boots, joins Wi-Fi by itself in about 2 s, and prints `EVT IP 172.20.10.2` (or similar) on USB serial.
- 3 Find the IP.** Read it with `python wdr_tool.py term --serial COM7` (press the board's EN button to see the boot lines), or look in the hotspot's list of connected devices.

- 4 Talk to it.** From a laptop on the same network:

```
python wdr_tool.py term --host 172.20.10.2
> PING
OK PONG
```

- 5 Prove it works.** `python bench_proof.py 172.20.10.2 COM7` runs 36 checks: commands, flash, reboot, and the pulse widths measured on the pins.

3 Pinout

ESP32 DEVKIT (THE COM7 BENCH)

FUNCTION	GPIO	HEADER LABEL
Channel 0 PWM	25	D25 / 25
Channel 1 PWM	26	D26 / 26
Channel 2 PWM	27	D27 / 27
Channel 3 PWM	33	D33 / 33
Ground	–	GND
USB serial (CP210x)	1 TX, 3 RX	TX0 / RX0
Reset / boot buttons	EN / 0	EN / BOOT

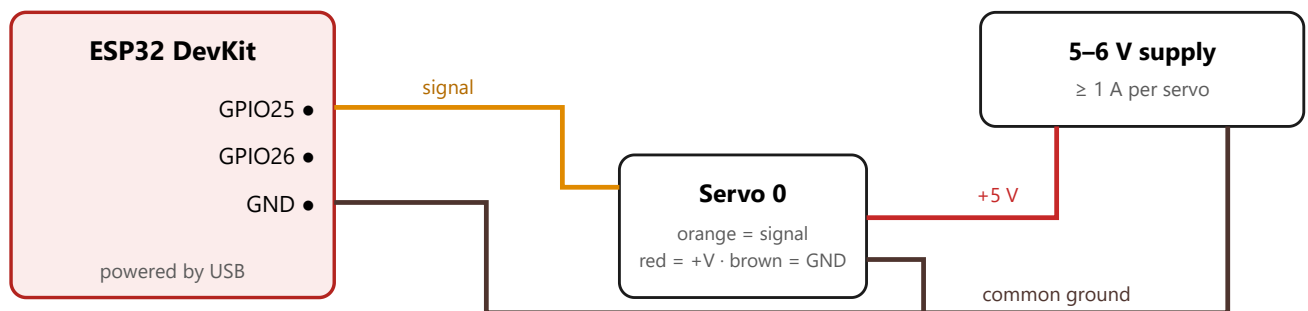
Never connect to GPIO 6–11 (labelled SD0–SD3, CMD, CLK). They are the flash chip's bus, and anything on them stops the board booting or being flashed.

7SEMI ESP32-S3 DEVKIT (COM6)

FUNCTION	GPIO	NOTES
Channel 0 PWM	4	
Channel 1 PWM	5	
Channel 2 PWM	6	
Channel 3 PWM	7	
Status RGB LED	48	on-board WS2812
USB serial (CH343)	43 TX, 44 RX	"USB for UART" port

Status LED on the S3: **dim blue** = STOPPED, **green** = RUNNING, **yellow** = PAUSED, **white flash** = cycle target reached. Never drive GPIO 26–37 on this board (flash and PSRAM). **The S3 build must be reflashed with the current firmware before use** (section 11).

4 Wiring servos



- **Signal** (orange or yellow) → the channel's GPIO.
- **Power** (red) → a **separate 5–6 V supply**, never the board's 3V3 pin. A moving servo draws up to 1 A and would reset the board.
- **Ground** (brown or black) → the supply's ground **and** a board GND pin. Without a shared ground the servo jitters.
- Most hobby servos accept the ESP32's 3.3 V signal directly. **Never feed 5 V into a GPIO.**
- **Demo without servos:** GPIO25 → 330 Ω → LED → GND. The brightness follows the profile (dimly, because a servo pulse is only 5–10 % of each 20 ms).
- After boot and after **STOP**, every output sits at **1500 μs** (servo centre).

5 Command reference

One command per line, ending in `\n`. Every command gets exactly **one** reply: `OK ...` or `ERR <CODE> ...`. "Allowed in" lists the states that accept the command. Anywhere else the reply is `ERR STATE`.

COMMAND	ALLOWED IN	REPLY	WHAT IT DOES
PING	any	OK PONG	Link check
INFO	any	OK INFO proto=1 team=WDR_REFERENCE ch=4 maxframes=8000 up=<s>	up = seconds since boot
SET <ch> <us>	STOPPED	OK	Put one output at a fixed width (0–3, 500–2500)
LOAD <rate> <frames>	STOPPED	OK	Start an upload (10–100 frames/s, 1–8000 frames). Deletes the stored profile
F <i> <us0> <us1> <us2> <us3>	after LOAD	OK	Frame <i>i</i> , sent in order from 0
COMMIT	all frames sent	OK SUM=<s>	Finishes the upload and saves it to flash. <i>s</i> = sum of all values mod 65536
START [cycles]	STOPPED	OK	Play from frame 0. <i>cycles</i> = target, omitted or 0 = forever
PAUSE	RUNNING	OK	Freeze. Outputs hold, counters are saved
RESUME	PAUSED	OK	Continue from the same frame
STOP	any	OK	All outputs to 1500 μ s, frame to 0, counters saved
STATUS	any	OK STATUS state=<S> cycle=<c> target=<t> frame=<f> frames=<n> us=<a,b,c,d>	Live state. <i>us</i> = what each pin outputs now
COUNTERS	any	OK COUNTERS cycles=<c> run_s=<r> active_s=<a,b,c,d>	Lifetime totals
CLEAR	STOPPED	OK	Reset all counters to 0
TEL <0 1>	any	OK	Telemetry stream off or on (off after boot and for each new client)
TIME <unix>	any	OK	Accepted for compatibility. Not used by this firmware
MEAS <ch>	any	OK MEAS ch=0 gpio=25 high_us=1500 period_us=20000 n=5	Self-test (this firmware only): measures the real pulse on the pin

6 Errors

CODE	MEANING
UNKNOWN	Not a command
ARGS	Wrong arguments, not a number, frame out of order, or line over 128 characters
RANGE	Channel, μ s, rate or frame count out of range
STATE	Not allowed in the current state
NOPROFILE	START with no profile
FULL	More than 8000 frames

7 Unprompted lines

LINE	WHEN
EVT BOOT	Firmware started (USB only)
EVT IP <addr>	Joined Wi-Fi (USB only)
EVT CYCLE <n>	A cycle finished
EVT DONE	Target reached, now STOPPED
TEL state=... cycle=... frame=... us=...	Every 500 ms after TEL 1

Lines starting with anything else are boot or debug text. Ignore them.

8 A typical run

```

> STOP
> LOAD 50 100
> F 0 1100 1300 1700 1900
> COMMIT
> TEL 1
> START 3

< OK
< OK          # 100 frames = 2 s per cycle
< OK          # ... up to F 99
< OK SUM=10176 # saved to flash now
< OK
< OK
< TEL state=RUNNING cycle=0 frame=24 us=1100,1300,1700,1900
< EVT CYCLE 1 ... EVT CYCLE 3
< EVT DONE    # outputs back to 1500 µs
< OK COUNTERS cycles=27 run_s=54 active_s=54,54,54,54
> COUNTERS

```

9 Behaviour you can rely on

SITUATION	WHAT THE BENCH DOES
Power cut or reset	Reboots in about 1 s and rejoins Wi-Fi in about 2 s. Comes back STOPPED with all outputs at 1500 µs, and never starts moving by itself . The counters lose at most the last 60 s of running. The profile is kept , so START works without a re-upload
Power cut during a save	The profile header is erased first and written last, so the bench comes back with either the old state or "no profile" (ERR NOPROFILE). It never keeps a half-written profile
Controller disconnects	Playback and counting carry on, on time. The next client to connect is served straight away
A second controller connects	The first one is disconnected. Only one controller at a time
Wi-Fi goes away	Keeps playing, retries joining with growing gaps (up to about 30 s), and prints EVT IP again when back
New upload	LOAD deletes the stored profile at once. If the upload isn't finished, there's no profile
START right after a large COMMIT	Waits until the flash save finishes (under 1 s for 4000 frames)
Frame rates above 50/s	Accepted, but a 50 Hz servo takes only one pulse per 20 ms, so at most 50 distinct frames per second reach the pins
Active time	active_s counts only while RUNNING and the channel is more than 25 µs from 1500
Timing accuracy	A hardware timer drives the frame clock on its own CPU core. Cycle period within $\pm 2\%$ (measured: exact) even while being flooded with commands

10 Tools (in `student_kit/`)

COMMAND	USE
<code>python wdr_tool.py term --host <ip></code>	Type commands by hand over Wi-Fi
<code>python wdr_tool.py term --serial COM7</code>	The same over USB. Shows <code>EVT IP</code> and boot lines
<code>python wdr_tool.py upload flight.csv --host <ip> --rate 50</code>	Upload a CSV (header <code>t_ms,ch0,ch1,ch2,ch3</code>)
<code>python wdr_tool.py makecsv flight.csv</code>	Make a sample 4-second profile
<code>python bench_proof.py <ip> COM7</code>	36-check end-to-end proof, including measured pulses and a reboot
<code>python wdr_tool.py test --host <ip> --serial COM7</code>	Full 85-point conformance scorer (about 4 min)
<code>python wdr_tool.py soak --host <ip> --minutes 30</code>	Long run that counts errors and timeouts
<code>python wdr_tool.py serve-sim</code>	A simulated bench on <code>127.0.0.1:3333</code> (no hardware)

Only one program can use a serial port at a time: close `idf.py monitor` or the Arduino Serial Monitor first. Only one controller can use the bench at a time: `term` kicks off any other client.

11 Building, flashing, changing Wi-Fi

Firmware source: `Hackathon/reference_fw/` (ESP-IDF v6.1). Open PowerShell and activate the toolchain first:

```
& 'C:\Espressif\tools\Microsoft.v6.1.PowerShell_profile.ps1'
cd C:\ESP32_Durability\Hackathon\reference_fw

# classic ESP32 bench (COM7)
idf.py -B build_esp32 -D SDKCONFIG=sdkconfig.esp32 -D IDF_TARGET=esp32 -p COM7 build flash

# ESP32-S3 bench (COM6)
idf.py -B build_s3 -D SDKCONFIG=sdkconfig.s3 -D IDF_TARGET=esp32s3 -p COM6 build flash
```

USE A DIFFERENT WI-FI NETWORK

Either give the router the default name `WELKINRIM-WDR` and password `welkinrim2026` (no rebuild needed), or change them and reflash:

```
idf.py -B build_esp32 -D SDKCONFIG=sdkconfig.esp32 menuconfig # WDR → Wi-Fi SSID / password
idf.py -B build_esp32 -D SDKCONFIG=sdkconfig.esp32 -p COM7 flash
```

The network must be 2.4 GHz with WPA2. On an iPhone hotspot, turn on "Maximise Compatibility". The hotspot name is the phone's name. If a flash fails with "serial data stream stopped", retry, or add `-b 115200`.

12 Troubleshooting

SYMPTOM	LIKELY CAUSE AND FIX
No EVT IP after boot	Network name or password doesn't match, the network is 5 GHz only, or the hotspot is hidden. Check section 11. Keep the iPhone's Personal Hotspot screen open while the board joins
Laptop can't connect to port 3333	Laptop and bench on different networks, a wrong IP, or campus Wi-Fi blocking device-to-device traffic. Ping the bench IP first
Connection closes by itself	Another client connected (one at a time). Close other term windows and servers
Telemetry stops after reconnecting	Normal: send TEL 1 again
Servo jitters or the board resets when a servo moves	The servo is powered from the board, or the grounds aren't joined. Use a separate 5–6 V supply with a common ground (section 4)
Servo doesn't move	Check MEAS <ch> : if it reports the right width, the problem is wiring or power, not the firmware
Board boot-loops, invalid header: 0xffffffff	Nothing on the flash, or the flash chip isn't answering. Unplug everything from GPIO 6–11 and reflash. Check with <code>python -m esptool --port COM7 flash-id</code> : a healthy board shows a real size such as 4MB, not Unknown
Serial port busy or access denied	Another program has the port open. Close monitors and terminals
START answers ERR NOPROFILE after a reboot	The last LOAD was never committed, or power went during a save. Upload again

13 Verified on the bench (25 Sep 2026)

END-TO-END PROOF: 36 / 36

Commands over Wi-Fi. Measured pulses 1000, 1100, 1300, 1500, 1700, 1900 and 2000 μ s, each within ± 2 μ s, period 20,000 μ s. **START**, **PAUSE** (outputs hold), **RESUME**, target $\rightarrow$ **EVT DONE** $\rightarrow$ idle. Reboot: **STOPPED**, counters and profile kept, plays again.

CONFORMANCE SCORER: 85 / 85

All 7 levels over Wi-Fi: USB hello, Wi-Fi link and takeover, manual PWM, upload, player timing ± 2 %, counters through a power cut, timing under a command flood, and play-through while the client is gone. **Known issue**: with the current firmware, two full runs scored 82/85. The three USB checks at the start of Level 1 intermittently got no reply. Level 1 scores 5/5 when run alone, and every other level scores full marks.